웹서비스설계 보고서 작성 양식

JWT 게시판 구현 보고서

01 서비스 소개와 구현 범위

학번 202212200 이름: 이대복 제출일: 2026.10.28

서비스 이름 오늘의 거지방

기술 조합과 버전 Vue / Spring Boot / SQLite / Java 21

GitHub 저장소 URL **https://github.com/rinda1234/wsd-crud-login-jwt**

소스 코드 기준 d5db2a589d6efe0a44f76406bb95ca4d612e6d6c

서비스의 대상 사용자와 사용 목적을 3~5 문장으로 설명하세요. 이 기술 조합을 선택한 이유와 제공 템플릿에서 변경한 범위를 적으세요.

오늘의 거지방은 사용자가 자신의 소비 내역이나 소비 경험을 게시글로 기록하여 하루의 소비를 돌아보고, 다른 사용자의 기록을 조회하여 바람직한 소비 습관을 배울 수 있는 웹 서비스이다. Vue 와 Spring Boot 를 선택한 이유는 실습을 통해 두 기술을 학습한 경험이 있이 비교적 익숙했기 때문이다. 기존에 학습한 내용을 바탕으로 프론트엔드와 벡엔드를 구성하였다.

기능	완료 여부	관련 파일	비고 또는 제한
공개 목록과 상세	완료	App.vue, BoardController.java 등	게시글 조회
로그인과 JWT 발급	완료	AuthService.java 등	로그인 성공 시 토큰 발급
인증과 내 정보 조회	완료	api.js	/api/me 요청
본인 글 작성·수정·삭제	완료	BoardController.java	작성자 권한 검사
만료·로그아웃·오류 표시	완료	api.js, App.vue	실제 동작 테스트 필요
DB 데이터 유지	완료	DatabaseConfig.java, PostStore.java 등	SQLite 파일 사용

구조와 변경 내역

브라우저 → API 서버 → SQLite 의 연결 구조를 간단히 그리거나 글로 설명하세요. 로그인 요청과 게시글 수정 요청의 흐름을 구분하고, 토큰이 어디에 저장되고 어느 요청에서 전송되는지 표시하세요.

본 서비스는 Vue 로 구성한 프론트엔드, Spring Boot 로 구성한 백엔드 API 서버, SQLite 데이터베이스로 구성된다. 프론트엔드는 사용자의 입력을 받아 백엔드에 API 요청을 보내고, 백엔드는 요청을 처리한 결과를 응답한다. 게시글과 사용자 데이터는 SQLite 데이터베이스를 통해 관리한다.
로그인할 때 사용자가 이메일과 비밀번호를 입력하면 프론트엔드는 로그인 API 에 인증 정보를 전송한다. 인증에 성공하면 서버에서 발급한 JWT 를 전달받아 프론트엔드의 session 변수에 저장한다. 이후 인증이 필요한 요청에는 Authorization: Bearer <토큰> 헤더를 포함한다.
게시글을 작성하거나 수정/삭제할 때 서버는 요청의 인증 정보를 확인하고, 해당 사용자가 작업을 수행할 권한이 있는지 검사한다. 로그아웃하면 프론트엔드에 저장된 로그인 상태를 초기화한다.

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
App.vue	로그인, 게시글 목록·상세 조회, 작성·수정·삭제 화면 제공	화면 제목과 안내 문구를 '오늘의 거지방' 콘셉트에 맞게 변경 (AI 참조)	서비스의 목적이 화면에 잘 드러나도록 하기 위해
style.css	화면의 글꼴, 색상, 여백, 버튼 및 카드 스타일 지정	배경과 카드, 입력창, 버튼 등의 색상과 디자인 변경(AI 참조)	소비 기록 서비스에 어울리는 화면을 구성하기 위해

로컬 실행 기록

운영체제 및 버전: Windows 11 Home 26H2

Java 버전: 21

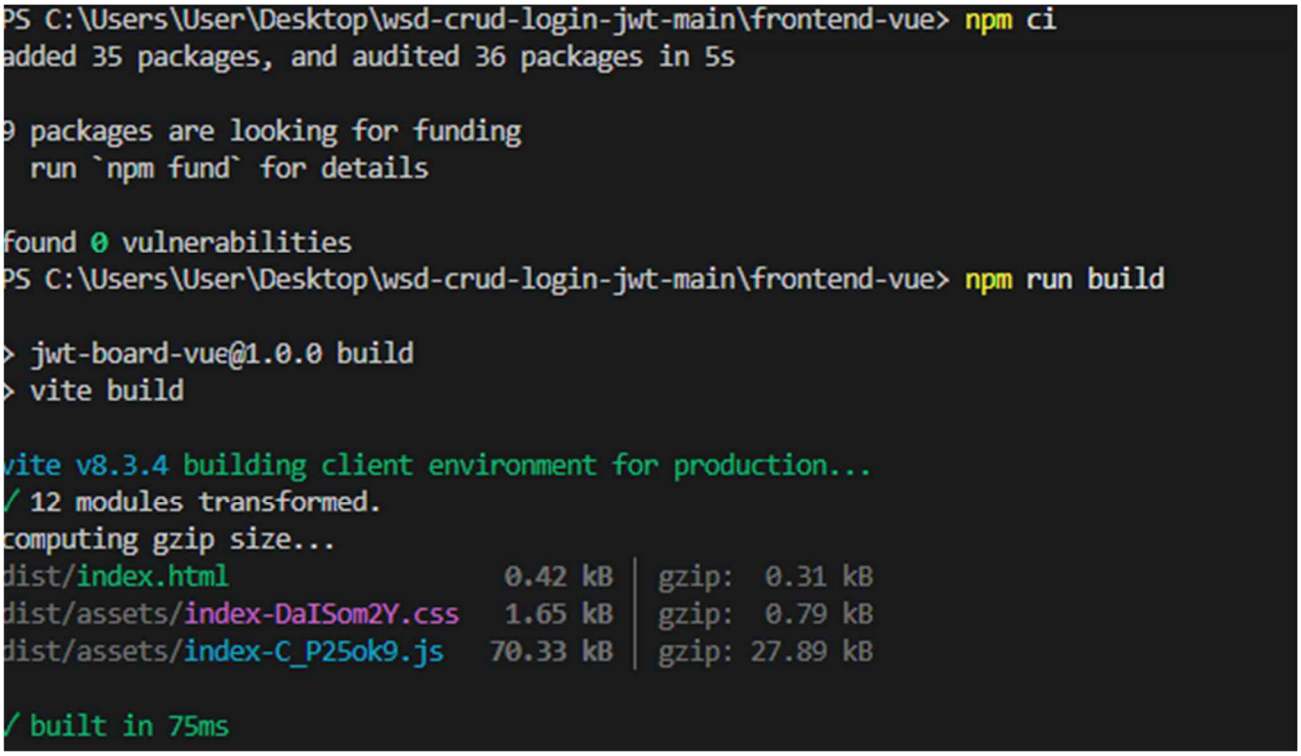
프론트엔드 설치 및 실행 명령:

Npm ci

Npm run dev

프론트엔드 빌드 명령 및 결과:

npm run build



백엔드 실행 명령 및 결과:

프론트엔드 접속 주소: http://127.0.0.1:4173/

API 연결 확인: [실제 요청 및 응답 결과 입력]

SQLite 파일 경로: [실제 설정된 경로 입력]

서버 재시작 후 데이터 유지 여부: [직접 확인한 결과 입력]

증거위치 : docs/evidence/backend.png, docs/evidence/frontend.png, health.png

웹서비스설계 03

핵심 코드 설명

3 1 로그인과 JWT 발급

파일 / 함수: AuthService.java / issueToken(), BoardController.java / login()

코드 발췌

```
@PostMapping("/auth/login")
 public Object login(@RequestBody Map<String, Object> body) {
  var input = fields(body, Map.of("email", 254, "password", 128));
  var user = users.verify(input.get("email"), input.get("password"));
  if (user == null) throw new ApiError(401, "INVALID_CREDENTIALS", "이메일 또는 비밀번호를 확인하세요.");
  return Map.of("accessToken", auth.issueToken(user), "expiresIn", ttl, "user", user);
 }
```

```
public String issueToken(Users.PublicUser user) {
    Instant now = Instant.now();
    return Jwts.builder().subject(Long.toString(user.id()))
      .issuer(issuer).audience().add(audience).and()
      .issuedAt(Date.from(now)).expiration(Date.from(now.plusSeconds(ttl)))
      .signWith(key, Jwts.SIG.HS256).compact();
  }
```

동작과 응답 코드 설명

"/auth/login" 경로로 들어온 Post 가 들어오면 login 함수가 실행된다. loginO 함수는 request 의 body 에서 email 과 password 를 가져오고, users.verify()를 호출하여 입력한 계정 정보가 올바른지 확인한다. 인증에 실패하면 HTTP 401 상태 코드와 INVALID_CREDENTIALS 메시지를 반환한다.

인증에 성공하면 auth.issueToken(user)을 호출하여 JWT 를 발급한다. issueToken()에서는 현재 시간을 기준으로 토큰 발급 시간과 만료 시간을 설정하고, userID 를 sub, 발급자를 iss, 토큰의 대상자를 aud 에 저장한다. 이후 서버의 비밀키를 이용하여 HS256 알고리즘으로 서명하고, compact()를 호출하여 설정한 정보를 JWT 문자열로 생성하여 반환한다.

이러한 방식으로 로그인에 성공한 사용자에게 토큰을 발급하고, 이후 API 요청에서 해당 토큰을 이용해 사용자를 인증할 수 있도록 하였다.

3 2 브라우저 토큰 저장과 전송

파일 / 함수: api.js/ login(), authorizationHeaders()

코드 발췌

```
export async function login(email, password) {
  const data = await request("/auth/login", {
    method: "POST",
    body: JSON.stringify({email, password})
  });
  session = {
    accessToken: data.accessToken,
    user: data.user,
  };
  return data.user;
}


function authorizationHeaders() {
if(!session.accessToken)
```

```
  return {};
return {
  "Authorization": "Bearer " + session.accessToken
 };
}
export function clearSession() {
 session = { accessToken: "", user: null };
}
```

동작과 응답 코드 설명

로그인에 성공하면 서버에서 받은 JWT 와 사용자 정보를 session 에 저장한다. 토큰은 accessToken 에 저장되며, 별도의 브라우저 저장소가 아니라 자바스크립트 메모리에서 관리한다.

이후 API 요청을 보낼 때 authorizationHeaders() 함수에서 토큰이 존재하는지 확인하고, 토큰이 있으면 HTTP 요청 헤더에 Authorization: Bearer + session.accessToken 형태로 추가한다. 이를 통해 서버는 요청을 보낸 사용자의 JWT 를 확인할 수 있다.

또한 API 응답이 401 상태 코드이면 clearSession()을 호출하여 저장된 토큰과 사용자 정보를 초기화한다.

서버 인증과 작성자 권한

3 3 서버 JWT 검증

파일 / 함수: AuthService.java / currentUserId()

코드 발췌

```
String header = request.getHeader("Authorization");

if (header == null || !header.startsWith("Bearer ")) throw new ApiError(401, "TOKEN_REQUIRED", "Bearer 형식 오류");

String token = header.substring(7);

if (token.isEmpty() throw new ApiError(401, "TOKEN_REQUIRED", "토큰 누락");

var jws = Jwts.parser().verifyWith(key).requireIssuer(issuer).requireAudience(audience).build().parseSignedClaims(token);

var claims = jws.getPayload();

if (!"HS256".equals(jws.getHeader().getAlgorithm()) || claims.getExpiration() == null)

   throw new IllegalArgumentException();

String sub = claims.getSubject();

if (sub == null || !sub.matches("[1-9][0-9]*") throw new IllegalArgumentException();

var userID = Long.parseLong(sub);

if (users.find(userID) == null) throw new IllegalArgumentException();

return userID;

} catch (ExpiredJwtException e) {

 throw new ApiError(401, "TOKEN_EXPIRED", "정상 서명의 토큰이 만료됨");

} catch (JwtException | IllegalArgumentException e) {

 throw new ApiError(401, "INVALID_TOKEN", "변조·서명·클레임 오류·없는 사용자");

}
```

동작과 응답 코드 설명

서버는 요청의 Authorization 에서 Bearer 뒤에 있는 JWT 를 추출한다. 이후 서버 키를 이용해 서명을 검증하고, 알고리즘이 HS256 인지 확인한다. 또한 토큰의 만료 여부와 발급자, 대상자, 사용자 ID 등의 정보를 검사한다. 마지막으로 토큰에 포함된 사용자 ID 가 실제로 존재하는지 확인하고, 검증이 완료되면 해당 ID 를 반환한다.

검증에 성공하면 정상적으로 요청을 처리하며, HTTP 응답 코드는 해당 API 의 처리 결과에 따라 결정된다.
검증에 실패하면 다음과 같이 오류를 반환한다.

- HTTP 401 / TOKEN_REQUIRED: 토큰이 없거나 Bearer 형식이 올바르지 않은 경우

- HTTP 401 / INVALID_TOKEN: 서명, 알고리즘, 클레임 또는 사용자 ID 검증에 실패한 경우
- HTTP 401 / TOKEN_EXPIRED: 토큰이 만료된 경우

이 과정을 통해 서버는 유효한 JWT 를 가진 사용자의 요청만 인증하고, 잘못된 토큰이나 만료된 토큰을 사용한 요청은 거부한다.

3 4 작성자 권한 검사

파일 / 함수: AuthService.java / assertOwner()

코드 발췌

```
 public void assertOwner(long userId, long authorId) {
   // TODO BE-03: 다른 작성자이면 403 FORBIDDEN. 현재는 검사하지 않습니다.
   if(userId != authorId) throw new ApiError( 403, "FORBIDDEN", "인증한 사용자가 다른 사람 글을 변경");
 }
}
```

동작과 응답 코드 설명

assertOwner()는 JWT 검증을 통해 확인한 현재 사용자 ID 인 userID 와 게시글에 저장된 작성자의 ID 인 authorID 를 비교한다. 두 ID 가 다르면 다른 사용자가 작성한 글을 조작하려는 것으로 판단해 403 FORBIDDEN 오류를 발생시킨다. 두 ID 가 같으면 오류를 발생시키지 않고 다음 작업을 진행한다.

401 Unauthorized 는 토큰이 없거나 유효하지 않아 사용자를 인증할 수 없는 경우에 발생한다. 반면 403 Forbidden 은 사용자가 정상적으로 인증되었지만 해당 게시글을 수정하거나 삭제할 권한이 없는 경우에 발생한다.

작성자 위조를 방지하려면 요청 본문에 포함된 authorId 를 그대로 신뢰하는 게 아닌, 서버에서 JWT 를 검증해 얻은 사용자 ID 를 기준으로 권한을 확인하고, 게시글의 실제 작성자 ID 와 비교해야 한다.

테스트 결과

실행 환경: [OS / 런타임 버전 / 브라우저 / 실행 명령]

로컬 배포 확인: [새 폴더에서 설치한 절차 / 프론트 빌드 결과 / 백엔드 실행 명령 / 4173 번 접속 및 API 연결 증거]

테스트 일시 및 코드 버전: [작성]

토큰 TTL: [일반 테스트 값 / 만료 테스트 값]

05_테스트시나리오.md 의 T01~T17 결과를 모두 기록하세요. PASS, FAIL, 미실행을 구분합니다. 예상 결과를 실제 결과로 복사하지 말고 직접 관찰한 HTTP 상태와 JSON code, 화면을 적으세요. 실제 결과 칸에는 예를 들어 403 / FORBIDDEN 처럼 기록합니다. 정상 응답은 200 / 해당 없음처럼 적습니다.

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T01	토큰 없이 목록 및 존재하는 글 상세 조회	200, 정상 공개 조회	200 / 해당없음	PASS	docs/evidence/T01.png
T02	Alice 계정으로 로그인	200, JWT 와 공개 사용자 정보, 화면 로그인 표시	200 / 해당없음	PASS	docs/evidence/T02.png
T03	잘못된 비밀번호 및 없는 이메일로 각각 로그인	401 INVALID_CREDENTIALS, 토큰 미발급	401 / Unauthorized	PASS	docs/evidence/T03_잘못된 비밀번호.png, docs/evidence/T03_잘못된 ID.png
T04	토큰 없이 /me 와 글 등록 요청	각각 401 TOKEN_REQUIRED	401 / Unauthorized	PASS	docs/evidence/T04_me.png, docs/evidence/T04_글등록.png
T05	Alice 토큰으로 글 작성 후 /me 조회	201, authorId 1, /me 는 Alice	200 / 해당없음	FAIL	docs/evidence/T05.png
T06	Alice 토큰으로 본인 글 수정·삭제	200 및 204, 결과가 DB 에 반영	200 / 해당없음, 204 / No Content	PASS	docs/evidence/T06_수정.png, docs/evidence/T06_삭제.png
T07	Alice 글을 Bob 의 정상 토큰으로 수정	403 FORBIDDEN, 원문 유지		미실시	[그림/파일명]
T08	Alice 글을 Bob 의 정상 토큰으로 삭제	403 FORBIDDEN, 글 유지	[실제]	미실시	[그림/파일명]

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T09	유효 토큰의 서명 부분 첫 글자를 다른 문자로 바꿔 /me 및 변경 요청	401 INVALID_TOKEN, 데이터 변경 없음	[실제]	미실시	[그림/파일명]
T10	짧은 TTL 로 발급하고 실제 만료 뒤 /me 및 변경 요청	401 TOKEN_EXPIRED, 화면 로그인 해제와 안내	[실제]	미실시	[그림/파일명]
T11	로그인 후 로그아웃, 다음 /me 요청	헤더에서 토큰이 빠지고 401 TOKEN_REQUIRED	[실제]	미실시	[그림/파일명]
T12	Bob 로그인 상태로 글 등록 요청에 authorId:1 또는 userId:1 추가	400 BAD_INPUT, 위조 글 미생성	[실제]	미실시	[그림/파일명]
T13	본인 토큰으로 빈 제목·공백 본문·길이 초과 입력을 각각 전송	400 BAD_INPUT, DB 변경 없음	[실제]	미실시	[그림/파일명]
T14	없는 양의 정수 ID 의 조회·수정·삭제 및 잘못된 ID 조회	없는 글 404, 잘못된 ID 400; 수정·삭제에는 정상 토큰 사용	[실제]	미실시	[그림/파일명]
T15	글을 만든 뒤 서버 종료·재실행	기존 글과 작성자·내용 유지	[실제]	미실시	[그림/파일명]
T16	로그인 후 페이지 새로고침	공개 글 유지, 메모리 로그인 상태 해제	[실제]	미실시	[그림/파일명]
T17	토큰 없는 수정·삭제 요청과 Bob 계정의 정상 작성·본인 수정·삭제	무토큰은 401, Bob 본인 작업은 성공	[실제]	미실시	[그림/파일명]

HTTP 응답 확인 기록

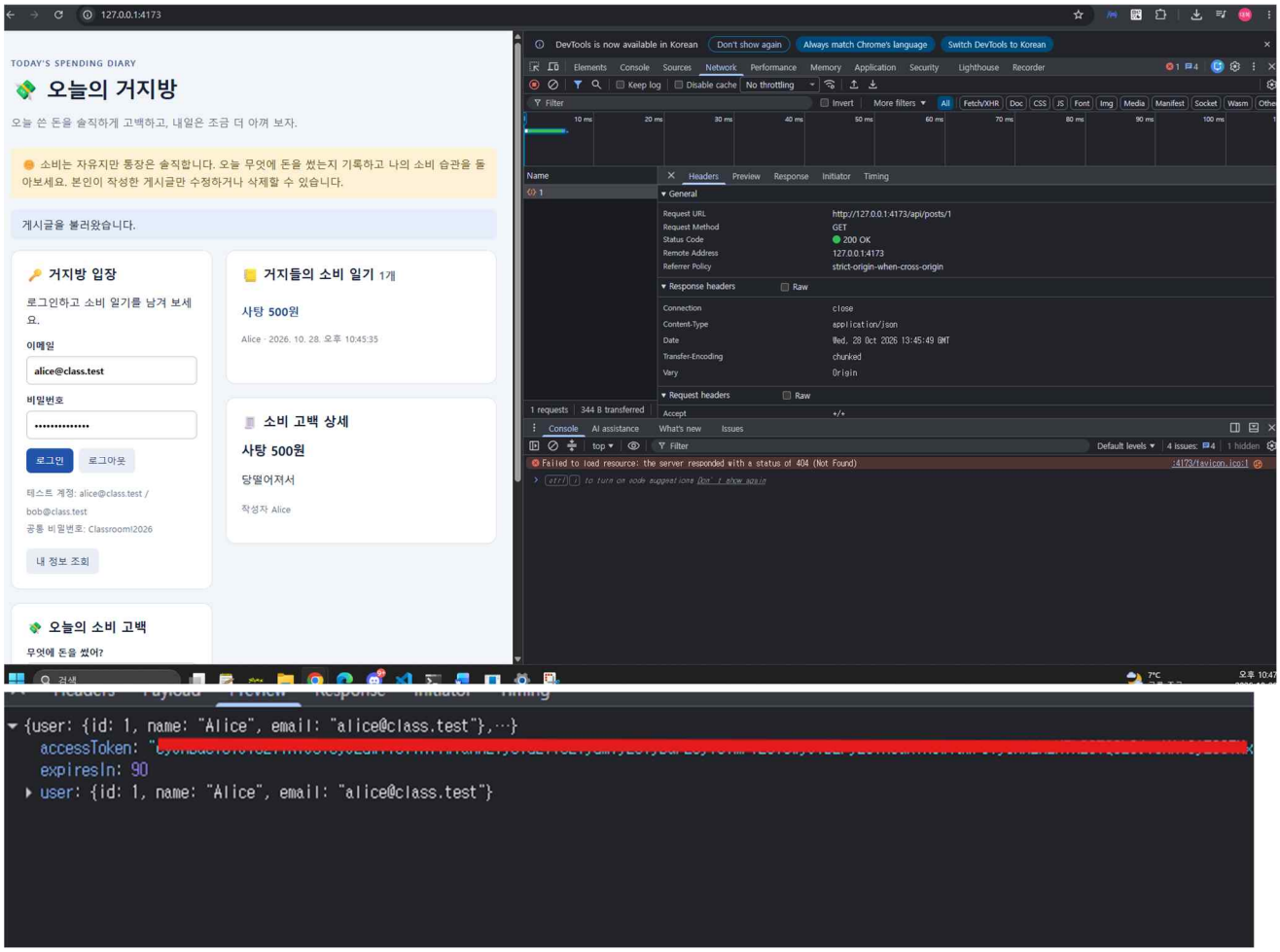
예상 코드와 실제 코드를 대조하고, 201·204·401 의 헤더와 본문도 확인하세요.

숫자는 Network 나 API 클라이언트에서 직접 관찰한 값을 기록합니다. 본문에 있는 숫자만으로 판단하지 않습니다. 한 항목이 여러 요청을 포함하면 행을 추가하세요.

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
정상 조회·로그인·수정	**200**	200 / 해당없음	docs/evidence/T01.png
새 글 등록	**201 및 Location**	200 / 해당없음	docs/evidence/T04_글등록.png
본인 글 삭제	**204 및 빈 본문**	204 / No Content	docs/evidence/T06_삭제.png
입력 오류·추가 작성자 필드	**400 BAD_INPUT**	[기록]	[기록]
잘못된 계정	**401 INVALID_CREDENTIALS**	401 / Unauthorized	docs/evidence/T03_잘못된 ID.png
토큰 없음·헤더 형식 오류	**401 TOKEN_REQUIRED**	401/TOKEN_REQUIRED	docs/evidence/토큰없음.png
변조·클레임 오류	**401 INVALID_TOKEN**	[기록]	[기록]
정상 서명의 만료 토큰	**401 TOKEN_EXPIRED**	401 TOKEN_EXPIRED"	docs/evidence/토큰만료.png
다른 작성자의 변경	**403 FORBIDDEN**	[기록]	[기록]
없는 글	**404 NOT_FOUND**	[기록]	[기록]

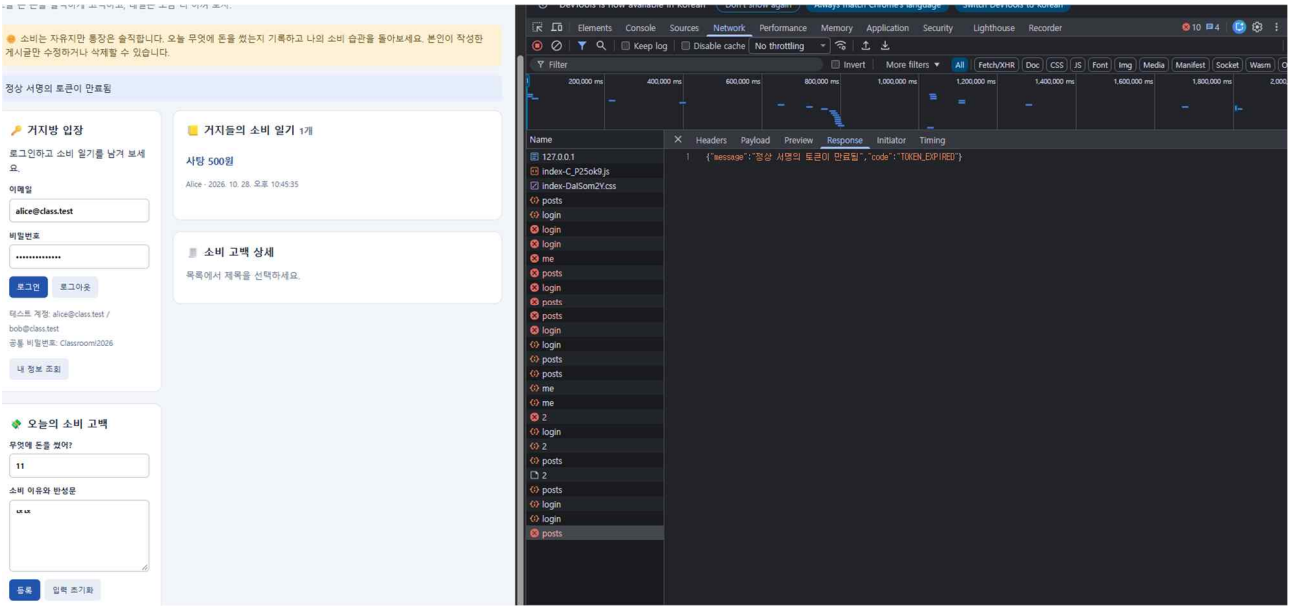
테스트 증거

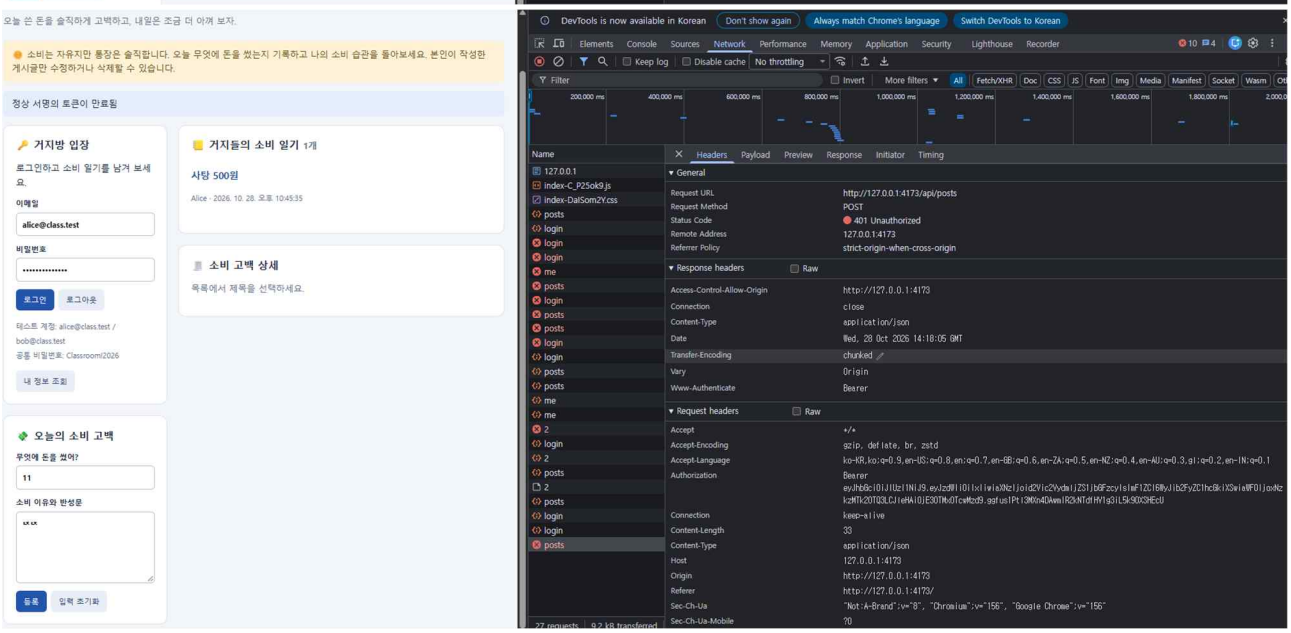
로그인과 토큰 전송



다른 작성자의 수정과 삭제

토큰 변조와 만료





자동 검사와 로컬 배포 증거

AI 활용과 참고 자료

AI 사용 여부: [사용 / 미사용]

사용하지 않았다면 미사용이라고 명시하고 참고한 수업 자료·공식 문서의 출처를 작성하세요. 사용했다면 전체 사용 범위를 요약하고 대표 사례 2~3 건을 작성하세요. 사용이 1 건이면 그 1 건만 적습니다. 대화 전체 제출은 필요하지 않습니다.

도구와 모델	사용 목적	반영한 파일 또는 보고서 항목	반영 방식
GPT-6	[학습/설계 /테스트/문장정리]	파일 전체	참고

대표 사례

도구 / 날짜: GPT-6

요청한 내용: 과제 요구사항과 프로젝트의 파일 구조를 파악하고, JWT 인증이 적용된 게시판을 구현하는 데 필요한 개념과 코드의 동작 원리를 이해하고자 AI 를 활용하였다.

AI 가 제안한 내용: 구현에 필요한 개념과 원리를 먼저 간략하게 설명하고, 이를 바탕으로 직접 코드를 작성할 수 있도록 도움을 받았다. 이후 작성한 코드를 함께 확인하면서 오류나 부족한 부분을 분석하고, 수정 방향을 검토하였다.

실제 반영 위치:

- JWT 로그인 및 토큰 발급
- 브라우저의 토큰 저장 및 요청 전송
- 서버의 JWT 검증
- 게시글 작성자 권한 검사
- 관련 백엔드 및 프론트엔드 코드

내가 확인하고 수정한 내용:

AI 의 설명을 바탕으로 개념을 이해한 뒤 직접 코드를 작성하였다. 작성한 코드를 다시 확인하며 각 함수의 역할과 동작 과정을 파악하고, 오류가 발생하거나 이해가 부족한 부분은 원인을 분석하여 수정하였다. 수정 과정에서 코드의 동작 원리를 이해하고 실제 구현에 맞는지 확인하는 데 중점을 두었다.

검증 방법과 결과: 작성한 코드의 동작을 확인하고, 필요한 경우 API 요청과 응답을 통해 인증 및 권한 검사 결과를 확인하였다.

참고 자료

이해도 확인과 회고

각 질문에 본인 코드의 동작을 근거로 답하세요.

1. 요청 본문의 authorId 를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

요청 본문에 있는 authorId 를 그대로 믿으면 다른 사용자가 자신의 ID 대신 다른 사람의 ID 를 보내 권한이 있는 것처럼 요청할 수 있다. 이를 막기 위해 서버에서 JWT 를 검증하여 현재 로그인한 사용자의 ID 를 가져오고, 게시글을 작성할 때 이 ID 를 작성자로 저장해야 한다. 수정이나 삭제 시에는 저장된 게시글 작성자 ID 와 인증된 사용자 ID 를 비교하여 서로 다르면 403 FORBIDDEN 오류를 발생시킨다.

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

수정 버튼을 숨기는 것은 화면에서만 접근을 제한하는 것이므로, 사용자가 API 요청을 직접 보내면 우회할 수 있다. 따라서 서버에서도 권한을 검사해야 한다. 내 코드에서는 AuthService.java 의 assertOwner() 함수가 인증된 사용자 ID 와 게시글 작성자 ID 를 비교하고, 서로 다르면 403 FORBIDDEN 오류를 발생시킨다.

3. JWT decode 와 verify 의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?

Decode 는 JWT 의 내용을 읽는 것이고, Verify 는 토큰의 서명 등을 검증하여 신뢰할 수 있는 토큰인지 확인하는 것이다. 내 코드의 currentUserId() 함수는 JWT 의 서명을 검증하고, 발급자와 대상, 만료 시간, 알고리즘, 사용자 ID 형식 및 해당 사용자의 존재 여부를 확인한다. 검증에 실패하거나 토큰이 만료되면 401 Unauthorized 오류를 발생시킨다.

4. 로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

로그아웃은 프론트엔드에서 session 에 저장된 토큰과 사용자 정보를 제거하여 이후 요청에 토큰이 포함되지 않도록 한다. 토큰 만료는 JWT 에 설정된 유효 기간이 지나 서버가 해당 토큰을 인증에 사용할 수 없게 되는 것이다. 새로고침은 페이지와 프론트엔드 상태를 다시 초기화할 수 있으며, 토큰을 메모리에만 저장했다면 새로고침 후 토큰이 사라져 다시 로그인해야 할 수 있다. 이미 복사된 토큰은 로그아웃만으로 서버에서 자동으로 무효화되는 것은 아니며, 만료되기 전까지는 서버의 검증을 통과할 가능성이 있다. 따라서 토큰이 유출되지 않도록 주의해야 한다.

5. 가장 어려웠던 오류 한 가지와 원인 확인·수정·재검증 과정을 설명하라.

개발 환경에서 백엔드 서버를 실행할 때 8000 번 포트가 이미 사용 중이어서 서버가 정상적으로 실행되지 않는 문제가 있었다. 원인을 확인하기 위해 netstat -ano | findstr :8000 명령어로 해당 포트를 사용하는 프로세스의 PID 를 확인하였다. 이후 해당 프로세스가 어떤 프로그램인지 확인하고, 포트 충돌을 해결하는 방법을 검토하였다. 수정 후에는 서버가 정상적으로 실행되는지와 API 요청이 가능한지를 다시 확인하는 방식으로 검증하였다.

현재 남은 문제와 다음에 개선할 점: 시간을 너무 촉박하게 잡아 보고서 작성 시간이 부족했음.

제출 확인: [ ] 실제 코드와 설명 일치 [ ] 테스트 직접 수행 [ ] AI 활용 및 출처 기록 [ ] 비밀 키와 토큰 제거

GitHub 확인: [ ] 루트 README 작성 [ ] 보고서 PDF 와 증거 업로드 [ ] README 문서 링크 확인 [ ] 교수자 접근 가능 [ ] 저장소 URL 과 커밋 SHA 제출